

1 Giai đoạn 2: PyTorch thành thạo (Becoming Fluent in PyTorch)

PHỤ THUỘC D/19/01-tu-viet-bang-numpy (bắt buộc — mọi khái niệm ở đây là tên gọi mới của thứ bạn đã cài bằng tay); B/11-gioi-han-tinh-toan (*roofline*, memory-bound so với compute-bound — không có nó thì mục về tăng tốc chỉ là danh sách mọ).

TÓM TẮT

Mục này đi theo mạch vấn đề → công cụ: mỗi tính năng của PyTorch được giới thiệu qua đúng cái bug hoặc cái nút thắt đã sinh ra nó, chứ không theo thứ tự tài liệu API. Đọc xong bạn đọc được một *profile* và biết nên bật *AMP*, sửa *DataLoader*, hay gọi `torch.compile` — thay vì bật cả ba rồi không biết cái nào có tác dụng. Nếu chỉ nhớ một điều: mọi công cụ tăng tốc ở đây chỉ giúp khi bạn đang nghẽn ở đúng chỗ nó gỡ, nên thứ tự bắt buộc luôn là *đo trước, sửa sau*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tensor** — mảng nhiều chiều kèm kiểu dữ liệu (`dtype`) và thiết bị (`device`, CPU hay GPU). Trong bộ nhớ nó chỉ là một dải liên tục cộng với bộ *stride* nói mỗi chiều nhảy bao nhiêu phần tử.
- **đồ thị tính toán (computation graph)** — bản ghi các phép toán đã thực hiện ở lượt xuôi, để lượt ngược biết đi ngược theo đường nào. PyTorch dựng nó *động*, tức mỗi lần lượt xuôi là một đồ thị mới.
- **epoch, batch, DataLoader** — một *epoch* là một lượt đi hết dữ liệu; *batch* là nhóm mẫu xử lý cùng lúc; *DataLoader* là bộ máy đọc đĩa, giải mã, augment và gom mẫu thành batch — thường chạy trên CPU và thường là chỗ nghẽn.
- **độ chính xác hỗn hợp (mixed precision)** — tính phần lớn phép toán ở 16 bit trong khi giữ trọng số và một số phép cộng dồn ở 32 bit. Mục đích là giảm số byte phải nạp qua bộ nhớ, không phải giảm số phép tính.
- **memory-bound so với compute-bound** — một kernel memory-bound bị chặn bởi băng thông bộ nhớ (làm ít phép tính trên mỗi byte nạp về), compute-bound thì bị chặn bởi năng lực tính toán. Hầu hết mạng thị giác là memory-bound — lý do AMP có tác dụng.
- **profiler** — công cụ đo thời gian thực sự của từng kernel và từng giai đoạn, thay vì đoán. Trong PyTorch là `torch.profiler`; ở tầng dưới là *Nsight*.
- **checkpoint và state_dict** — `state_dict` là từ điển chứa toàn bộ trọng số và *buffer* của mô hình; lưu nó ra đĩa được gọi là checkpoint. Nó *không* chứa mã nguồn, nên vẫn cần đúng phiên bản code để nạp lại.
- **tính tất định (determinism)** — tính chất chạy lại cùng đầu vào cho ra cùng đầu ra tới từng bit. Trên GPU nó không mặc định, và ép nó thì phải trả giá bằng tốc độ.

A. Đặt vấn đề

Sau giai đoạn 1 bạn biết cơ chế nhưng chưa có công cụ. PyTorch cung cấp công cụ, nhưng nó cung cấp *rất nhiều* công cụ, và tài liệu chính thức tổ chức theo API chứ không theo vấn đề. Hệ quả là kiểu hiểu biết phổ biến nhất: biết gọi `amp.autocast` mà không biết nó giúp gì; bật `torch.compile` rồi thấy không nhanh hơn và kết luận rằng “compile không ăn thua”.

Mục này đảo lại thứ tự trình bày. Mỗi công cụ xuất hiện sau đúng cái vấn đề đã sinh ra nó, vì trong nghề, thứ bạn thực sự cần nhớ không phải tên hàm — tra được trong ba mươi giây — mà là *ánh xạ từ triệu chứng sang công cụ*. Cần nói rõ đây là nội dung tầng bốn [T4]: tên API sẽ đổi, và điều đó bình thường; cái không đổi là danh sách các nút thắt.

→ **tiền đề** D/19/01 — *autograd*, *buffer* và cặp *train/eval* chỉ là tên gọi mới của đồ thị tính toán, trạng thái và công tắc chế độ mà bạn đã tự cài bằng tay.

Học xong thì làm được gì? Bạn cầm một vòng lặp huấn luyện chậm gấp ba lần đáng ra, chạy *profiler*, và chỉ đúng một dòng cần sửa — đây chính là kỹ năng bạn đã có với *nsys* và *perf* ở phía C++/CUDA, lần này áp cho một *runtime* có thêm một tầng trừu tượng ở giữa.

TRỰC GIÁC

Autograd là một cuốn băng ghi âm chỉ dùng được một lần. Mỗi phép toán trên *tensor* có `requires_grad=True` ghi thêm một mục vào băng; `backward()` tua ngược băng, tính gradient, rồi *xoá* băng đi. Từ hình ảnh đó suy ra ba điều mà người ta thường phải học bằng cách vấp: gọi `backward()` hai lần thì lỗi vì băng đã bị xoá (trừ khi bạn xin `retain_graph`); giữ một *tensor* còn dính băng trong một danh sách log nghĩa là giữ luôn toàn bộ đoạn băng phía sau nó, nên bộ nhớ phình dần suốt *epoch*; và `detach()` chỉ đơn giản là cắt băng ở đúng chỗ đó.

B. Mạch vấn đề → công cụ

I Chín nút thắt, theo thứ tự bạn sẽ gặp

- 1 Vấn đề** bug im lặng về bộ nhớ và tính đúng đắn.

Giải pháp hiểu *tensor* ở mức bố cục bộ nhớ: `dtype`, `device`, *stride* (mỗi bước theo một chiều nhảy bao nhiêu phần tử trong bộ nhớ), và khác biệt *view* với *copy*.

Hệ quả `.view()` thất bại trên *tensor* không liên mạch, còn `.reshape()` âm thầm sao chép — một `.contiguous()` đặt sai chỗ có thể nhân đôi bộ nhớ mà không báo gì.
- 2 Vấn đề** không biết đồ thị được dựng và giải phóng lúc nào.

Giải pháp *autograd*: `backward`, `no_grad`, `detach`, và `autograd.Function` tùy biến khi bạn có công thức gradient riêng.

Vấn đề mới bug kinh điển: giữ *tensor* còn đồ thị trong một list để logging, làm rò bộ nhớ suốt *epoch*.
- 3 Vấn đề** trạng thái mô hình không chỉ là trọng số.

Giải pháp `nn.Module`: phân biệt *parameter* với *buffer*, hiểu `state_dict`, dùng *hook*, và cập `train()/eval()`.

Giá định bạn nhớ rằng thống kê chạy của `BatchNorm` là *buffer* chứ không phải tham số — nó được lưu, được chuyển `device`, nhưng không được *optimizer* đụng tới.
- 4 Vấn đề** GPU đói dữ liệu.

Giải pháp `Dataset/DataLoader/sampler`, `num_workers`, `pin_memory`, *prefetch*, `collate_fn`; và một tầng *augmentation* (`albumentations`, `transforms v2`, hoặc `DALI` khi CPU không kịp).

Hệ quả đây là nút thắt phổ biến nhất và cũng bị bỏ qua nhiều nhất — xem B/11-gioi-han-tinh-toan.
- 5 Vấn đề** framework giấu quá nhiều.

Giải pháp tự viết vòng lặp huấn luyện ít nhất một lần trước khi dùng *Lightning* hay *HF Trainer*.

Giá phải trả vài giờ.

Hệ quả sau đó dùng framework là hợp lý, vì bạn đã biết nó đang làm gì và biết chỗ để chọc vào khi nó làm sai.

6 Vấn đề

Giải pháp

bộ nhớ và tốc độ.

AMP (tính phần lớn phép toán ở 16 bit, giữ trọng số ở 32 bit) với GradScaler; **DDP** (chia batch cho nhiều GPU, mỗi GPU giữ một bản sao mô hình) qua `torchrun`, hoặc FSDP khi một GPU không chứa nổi mô hình [1]; và `torch.compile` để hợp nhất kernel và giảm chi phí điều phối.

7 Vấn đề

Giải pháp

Giả định

đo trước khi tối ưu.

`torch.profiler`, `Nsight`, `trace viewer`.

bạn tin số đo hơn tin trực giác — nguyên tắc đã có ở B/11-gioi-han-tinh-toan.

8 Vấn đề

Giải pháp

Giá phải trả

kết quả không lặp lại.

`seed`, `torch.use_deterministic_algorithms`, tắt `cudnn.benchmark` — kèm cái giá về tốc độ.

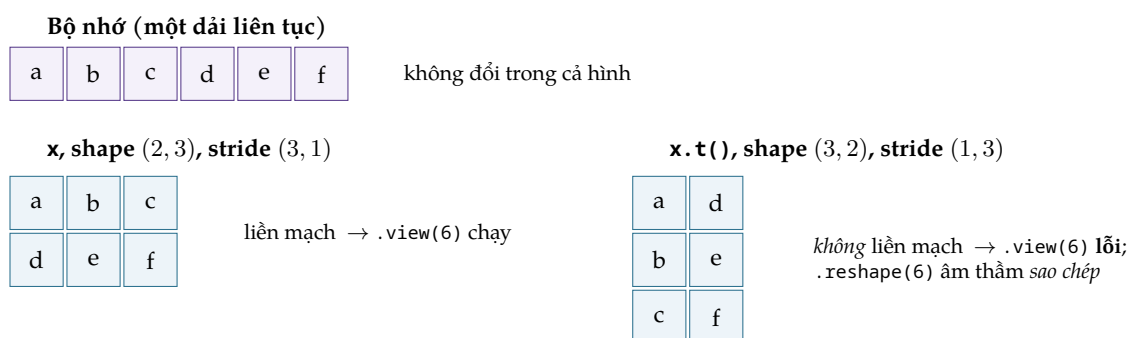
đáng kể, và *vẫn không* đảm bảo tái lập giữa hai máy khác GPU. Toàn bộ chủ đề này thuộc D/22-tai-lap-va-mlops.

Hệ quả

biết là đủ: viết kernel riêng bằng CUDA hoặc Triton. Cần khi bạn có một mẫu truy cập bộ nhớ đặc thù mà không kernel nào khớp — hiếm, và thường có cách rẻ hơn.

C. Stride, view và copy — bố cục bộ nhớ là thứ thật

Đây là chỗ người từ C++ có lợi thế lớn nhất, vì **stride** chính là thứ bạn vẫn dùng khi viết kernel: một tensor là một con trỏ, một **shape**, và một bộ **stride** nói mỗi bước theo mỗi chiều nhảy bao nhiêu phần tử. Hình 1 cho thấy vì sao chuyển vị không tốn gì và vì sao `.view()` sau đó lại thất bại.



Hình 1. Chuyển vị chỉ đổi **stride**, không chạm vào bộ nhớ — nên nó gần như miễn phí. Nhưng `.view()` đòi hỏi các phần tử nằm đúng thứ tự trong bộ nhớ, nên nó thất bại; `.reshape()` thì không thất bại mà lặng lẽ cấp phát một bản sao. Chính sự “lặng lẽ” đó là nguồn của các lần bùng nổ bộ nhớ không giải thích được.

Quy tắc thực hành rút ra: dùng `.view()` khi bạn *muốn* chương trình báo lỗi nếu bố cục không như bạn nghĩ, và dùng `.reshape()` chỉ khi bạn đã chấp nhận cái giá sao chép. Đây là cùng một triết lý với `at()` so với `operator[]` trong C++: chọn phiên bản ồn ào ở chỗ bạn chưa chắc.

D. Parameter, buffer và cặp train/eval

Ở giai đoạn 1 bạn đã phải tự trả lời “cái gì là tham số, cái gì là trạng thái”. PyTorch đặt tên cho câu trả lời đó:

- **Parameter** — được optimizer cập nhật, có gradient, nằm trong `state_dict`. Ví dụ: γ , β của BatchNorm.

- **Buffer** — không có gradient và optimizer không đung tới, nhưng vẫn nằm trong `state_dict` và vẫn theo mô hình khi bạn gọi `.to(device)`. Ví dụ: `running_mean`, `running_var` của Batch-Norm.
- **Biến tạm** — không lưu, không chuyển; nếu bạn cần nó tồn tại qua một lần lưu checkpoint thì bạn đã chọn sai loại.

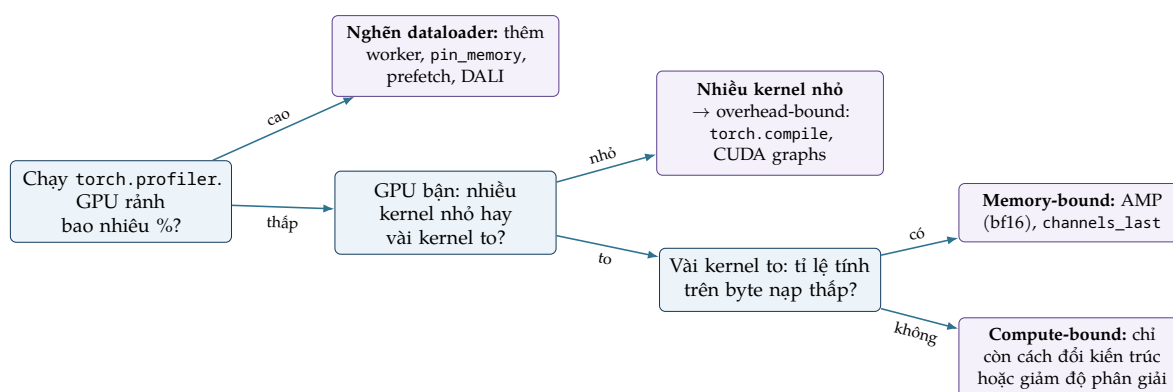
Từ đó suy ra vì sao quên `model.eval()` là lỗi kinh điển và im lặng: ở chế độ train, BatchNorm dùng thống kê của *batch hiện tại* thay vì buffer, và dropout bật lại. Kết quả là đầu ra phụ thuộc vào việc bạn tình cờ đưa mẫu nào vào cùng batch — một mẫu đánh giá có thể ra hai kết quả khác nhau chỉ vì bạn đổi thứ tự dữ liệu. Không có exception nào; chỉ có một con số hơi lệch.

↪ **hệ quả của C/12** — BatchNorm được đưa vào để ổn định huấn luyện; chính cơ chế ấy làm mô hình phụ thuộc cả batch, nên nó kéo theo hai chế độ hành vi khác nhau.

E. Tăng tốc: đo trước, và biết mỗi đòn bẫy gỡ nút thắt nào

Đây là phần bị lạm dụng nhiều nhất, nên bắt đầu bằng nguyên tắc: *mỗi đòn bẫy chỉ giúp khi bạn đang nghẽn ở đúng chỗ nó gỡ*. Hình 2 là cây quyết định rút gọn, và Bảng 1 là phần chi tiết.

↪ **ràng buộc bởi B/11** — cây quyết định tăng tốc chỉ là mô hình roofline diễn đạt bằng tên API: nhánh nào đúng phụ thuộc bạn đang bị chặn bởi băng thông, năng lực tính, hay chi phí điều phối.



Hình 2. Cây quyết định tăng tốc một vòng lặp huấn luyện. Nhánh đầu tiên — GPU đang rảnh — là nhánh đúng trong phần lớn trường hợp thực tế, và cũng là nhánh mà mọi mẹo về `torch.compile` đều vô dụng.

Bảng 1. Năm đòn bẫy tăng tốc, và nút thắt mà mỗi cái gỡ.

Đòn bẫy	Cơ chế	Gỡ nút thắt nào	Hỏng / vô dụng khi nào
Sửa DataLoader	Nhiều <i>worker</i> , <i>pin_memory</i> , <i>prefetch</i> , chuyển augmentation lên GPU	GPU đói dữ liệu	Quá nhiều worker làm cạn RAM và tăng thời gian khởi động; augmentation trên GPU tranh chấp với training
AMP (fp16/bf16)	Nửa số byte phải nạp và ghi	Memory-bound	Compute-bound thuận; fp16 tràn số khi không có GradScaler
<i>channels_last</i>	Đổi bố cục sang NHWC cho <i>Tensor Core</i>	Conv memory-bound	Mô hình có op không hỗ trợ NHWC → thêm phép hoán vị, chậm hơn
<code>torch.compile</code>	Hợp nhất kernel, giảm chi phí điều phối Python	Overhead-bound	Shape động gây biên dịch lại liên tục; <i>graph break</i> do code Python động
DDP / FSDP	Chia batch (DDP) hoặc chia trọng số (FSDP) qua nhiều GPU	Không đủ thời gian hoặc bộ nhớ	Băng thông liên kết kém → <i>all-reduce</i> nuốt hết lợi ích; batch hiệu dụng đổi nên phải chỉnh <i>learning rate</i> [2]

Ví dụ nhỏ nhất cho AMP. Một tensor kích hoạt cỡ $64 \times 256 \times 56 \times 56$ có 51,4 triệu phần tử. Ở fp32 nó chiếm 205 MB; ở bf16 chiếm 103 MB. Trên một GPU có băng thông 900 GB/s, chỉ riêng việc đọc nó một lần tốn $228 \mu\text{s}$ so với $114 \mu\text{s}$. Ý nghĩa: nếu tăng đó chỉ làm vài phép tính trên mỗi byte nạp về, AMP cắt gần một nửa thời gian; nếu nó làm rất nhiều phép tính trên mỗi byte, AMP gần như không đổi gì. Đó chính là trục *arithmetic intensity* của B/11-gioi-han-tinh-toan, xuất hiện lại dưới dạng một lựa chọn hai dòng code.

fp16 hay bf16? Hai lựa chọn đối lập rõ ràng nên đáng đặt cạnh nhau: fp16 có 10 bit phần định trị và dải mũ hẹp, nên chính xác hơn ở vùng giá trị bình thường nhưng tràn hoặc triệt tiêu về không khi gradient nhỏ — vì thế nó *bắt buộc* cần GradScaler để nhân loss lên trước khi lượt ngược [3]. bf16 có cùng dải mũ với fp32 nhưng chỉ 7 bit định trị: kém chính xác hơn, đổi lại gần như không bao giờ tràn và không cần GradScaler. Trên phần cứng hỗ trợ bf16, mặc định hợp lý là bf16; fp16 chỉ còn lý do tồn tại trên phần cứng cũ.

```
scaler = torch.amp.GradScaler() # chỉ cần cho fp16
model = model.to(memory_format=torch.channels_last)
for x, y in loader:
    x = x.cuda(non_blocking=True).to(memory_format=torch.channels_last)
    opt.zero_grad(set_to_none=True) # set_to_none re hon zero_
    with torch.amp.autocast("cuda", dtype=torch.bfloat16):
        loss = criterion(model(x), y.cuda(non_blocking=True))
    scaler.scale(loss).backward()
    scaler.step(opt); scaler.update()
```

F. Giới hạn và trường hợp hỏng

- **torch.compile không phải nút bấm thần kỳ.** Nó giúp khi bạn đang overhead-bound; nó không giúp gì cho mô hình nghẽn ở dataloader, và nó có thể *chậm hơn* nếu shape đầu vào thay đổi liên tục khiến nó biên dịch lại. Với dữ liệu ảnh có kích thước thay đổi hoặc số hộp phát hiện thay đổi, đây là tình huống thường gặp chứ không phải ngoại lệ.
- **Profiler cũng nói dối nếu bạn đo sai.** Lệnh CUDA là bất đồng bộ, nên đo bằng đồng hồ Python quanh một lời gọi chỉ đo thời gian *xếp hàng*, không đo thời gian chạy. Phải đồng bộ hoá hoặc dùng CUDA event — một cái bẫy bạn đã gặp ở phía C++, và nó không biến mất khi lên Python.
- **DDP đối ngữ nghĩa của batch.** Batch hiệu dụng là batch mỗi GPU nhân số GPU, nên mọi siêu tham số phụ thuộc batch (learning rate, lịch warm-up, thống kê BatchNorm) đều đổi theo. Một so sánh “1 GPU với 8 GPU” mà không chỉnh learning rate là một so sánh vô nghĩa.
- **Tắt định có giá.** Bật use_deterministic_algorithms buộc PyTorch chọn kernel chậm hơn hoặc báo lỗi ở những op không có bản tắt định — và nó vẫn không đảm bảo hai máy khác GPU cho cùng kết quả.

CẠM BÃY

Rò bộ nhớ vì logging là bug tốn thời gian nhất trong danh sách này, vì triệu chứng xuất hiện sau vài trăm bước chứ không phải ngay lập tức. Viết `losses.append(loss)` thì bạn đang giữ luôn toàn bộ đồ thị tính toán dẫn tới loss, kể cả mọi activation trung gian; sau một epoch, GPU hết bộ nhớ và *stack trace* chỉ vào một dòng hoàn toàn vô can. Cách đúng là `losses.append(loss.detach())` hoặc `loss.item()`. Cùng một họ lỗi: giữ output của mô hình trong một *dict* để debug, hoặc trả tensor còn đồ thị ra khỏi một hàm được gọi trong vòng lặp.

CẦN NHỚ

Ảnh xạ triệu chứng → công cụ mới là thứ đáng nhớ, không phải tên API. GPU rảnh thì sửa dataloader; nhiều kernel nhỏ thì `torch.compile`; ít tính trên mỗi byte thì AMP và `channels_last`; không đủ bộ nhớ thì FSDP hoặc *gradient checkpointing*. Và trước tất cả: chạy profiler, vì tối ưu thứ chưa đo là cách chắc chắn nhất để mất một tuần mà không nhanh hơn. [T2]

G. Kết nối

Mục này là bản dịch của D/19/01-tu-viet-bang-numpy sang ngôn ngữ công cụ: *autograd* là cuốn băng bạn đã tự cuộn bằng tay, *buffer* là “trạng thái” bạn đã tự quản lý, `eval()` là cái công tắc bạn đã tự viết. Nó là ứng dụng trực tiếp của B/11-gioi-han-tinh-toan: cây quyết định ở Hình 2 chỉ là mô hình *roofline* được diễn đạt bằng tên API. Nó là tiền đề của D/20-toi-uu-va-nen-mo-hinh (nơi cùng những đòn bẩy này được áp cho **suy luận** (*inference*) thay vì huấn luyện) và của D/22-tai-lap-va-mlops (nơi mục về *seed* và tính tất định được mở rộng thành một chương).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Khi nào `.view()` thất bại mà `.reshape()` thì không, và điều đó liên quan gì tới *stride*? Vì sao thất bại đôi khi lại là hành vi bạn muốn?
2. Vì sao `torch.compile` không giúp gì cho mô hình bị nghẽn ở dataloader, và làm sao biết mình đang ở tình huống đó trong ba mươi giây?
3. Thống kê chạy của BatchNorm là *parameter* hay *buffer*, và câu trả lời đó giải thích thế nào cho việc quên `eval()` lại làm kết quả phụ thuộc thứ tự dữ liệu?
4. Vì sao fp16 cần GradScaler mà bf16 thì không? Cái giá của bf16 là gì?
5. Bạn đo một lời gọi mô hình bằng `time.time()` và thấy nó tốn 0,2 ms. Vì sao con số đó gần như chắc chắn sai, và phải đo thế nào?
6. Chuyển từ 1 GPU sang 8 GPU bằng DDP, mọi thứ chạy nhưng độ chính xác tụt — hai nguyên nhân khả dĩ nhất là gì, biết rằng cả hai đều không phải bug trong DDP?

Nguồn

- [1] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [2] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [3] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.